Kristianstad University
SE-291 88 Kristianstad
Sweden
+46 44 250 30 00
www.hkr.se

Analysis report

Varvara Aladyina

November 2025

Title
Analysis project for sustainable development

Programme
Software Development

Author
Varvara Aladyina

Abstract
prioritizes testability.

Keywords
TDD, Test Driven Development, Unit testing, functional paradigm, FP, Cohesion, Technical debt, Sustainable Development

Contents

1 Actual algos **4**

2 Introduction **5**

3 Methodology **5**
3.1 Analytical methods 5
3.2 Practical methods 5

4 Result **6**
4.1 Coverage.py 6
4.2 Maintainability index 6
4.3 Halstead Metrics 7
4.4 Cohesion 8

5 Discussion **9**
5.1 Philosophies and Principles 9
5.2 Test Driven Development and cost of the proccess flaw 9
5.2.1 Team's approach to the assignment and how TDD wasn't implemented 9
5.3 Functional Programming 10
5.3.1 Pure Functions in Unit Testing 10
5.3.2 Functional paradigm or conventional style 10
5.4 Technical debt 10

6 Summary **10**

7 Bibliography **11**

A Reflections on Polyglot Programming **12**

B Sustainable development **12**

1 Actual algos

Hello :) this is algos!

2 Introduction

The definition of "good code" is often debated. Every line of code represents a cost of something, whether it's money or time for proper maintenance and readability. According to Loup Vaillant [1] good code is characterized by:

- Possibility of writing code as little as possible (less lines of code means less reading and less time spent maintaining)
- Code doesn't have many bugs
- Code shouldn't use many resources
- Code should not perform slow

To write a good code programmer should have spent as minimum time as possible on prerequisite knowledge (with what program will do, libraries, frameworks, etc.), knowledge of existing code and what it does, and coordination (working efficiently in team).

This report reflects on a group software development project designed to explore these concepts practically. The project aimed to create a dice game using Object-Oriented Programming (OOP) in Python, using Test Driven Development, unit testing and conventional styles.

This paper covers my experience of doing group project. The objectives of this paper are:

- Test Driven Development in theory and practise.
- Impact of Functional Paradigm on Unit Testing.
- Assess code sustainability using cohesion, cyclomatic complexity and maintability index.

3 Methodology

The report based both on analytical and practical methods.

3.1 Analytical methods

Before and after assignment, literature review was made on functional programming paradigms, unit testing strategies, and sustainable software principles. This theoretical framework provides the baseline for evaluating the team's performance.

3.2 Practical methods

Data was harvested from the group project repository hosted on GitHub. The following tools were used to generate objective quality metrics:

- Coverage.py [2] measured the extent to which unit tests covered project.
- Radon measured Cyclomatic Complexity (CC) and Maintainability Index (MI), which are the keys for clean and sustainable code. CC is useful metrics to understand how complex is the code and MI measures how easy is it to maintain the code. MI includes different metrics, such as CC and Halstead volume that will be in this paper.

4 Result

Folowing data shows objectively quality of the codebase of the project.

4.1 Coverage.py

This is how coverage.py rated project of my group team:

```
tests\test_dice.py .......                                  [  9%]
tests\test_game.py ...................                      [ 37%]
tests\test_high_score.py .......                            [ 47%]
tests\test_menu.py .............                            [ 66%]
tests\test_player.py ......................                 [100%]

Name                     Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------
program\__init__.py          0      0      0      0   100%
program\computer.py        137    120     60      0     9%   27-32,36-37,41,45,
                                                              54-68,81-107,117-145,
                                                              154-168,181-198,
                                                              209-231, 234, 237
program\dice.py             13      0      4      0   100%
program\dice_hand.py        26     19     10      0    19%   9-11,19-21,29-33,40-43,
                                                              50-53,61
program\game.py             59     31     24      2    43%   45-78, 81-83, 86-87,
                                                              97->exit, 152-154
program\high_score.py       20      1      6      1    92%   49
program\menu.py             96     43     32      6    54%   75,93->114,97-98,
                                                              103-104,109-110,
                                                              131-137,160,168,
                                                              178-190,199-212,
                                                              233,246,254,
                                                              257-276,280-283
program\player.py           54     26     14      0    47%   47-68, 71-79, 113
-----------------------------------------------------------------------
TOTAL                      405    240    150      9    36%
```

The coverage.py analysis revealed significant gaps in testing. The total project coverage was recorded at 36%, which is extremely low for a sustainable project.It highlights that program/computer.py, the most complex logic handler, had the lowest coverage (9%).

4.2 Maintainability index

This information shows maintability of the project calculating with cyclomatic complexity:

```
radon mi --show program
program\computer.py - A (53.72)
program\dice.py - A (83.67)
program\dice_hand.py - A (82.73)
```

```
program\game.py - A (68.38)
program\high_score.py - A (81.58)
program\menu.py - A (64.62)
program\player.py - A (69.96)
program\__init__.py - A (100.00)
```

The Radon analysis for Maintainability Index classified all files as 'A' grade, however the computer.py is still with lowest numbers bordering with 'B' grade.

- `program/dice.py`: 83.67 (High maintainability)
- `program/high_score.py`: 81.58
- `program/computer.py`: 53.72 (Lowest score, bordering on 'B')

4.3 Halstead Metrics

Table 1: Halstead Metrics for Program Modules

File	h1	h2	N1	N2	Voc.	Len.	Calc.Len.	Volume	Diff.	Effort
program\computer.py	12	62	45	84	74	129	412.18	801.02	8.13	6511.51
program\dice.py	2	4	3	5	6	8	10.00	20.68	1.25	25.85
program\dice_hand.py	2	7	4	7	9	11	21.65	34.87	1.00	34.87
program\game.py	5	12	8	14	17	22	54.63	89.92	2.92	262.28
program\high_score.py	4	12	8	14	16	22	51.02	88.00	2.33	205.33
program\menu.py	6	17	12	20	23	32	85.00	144.75	3.53	510.90
program\player.py	5	18	11	22	23	33	86.67	149.28	3.06	456.13
program__init__.py	0	0	0	0	0	0	0.00	0.00	0.00	0.00

Where:

- h1 stands for **Unique Operators** (η_1)
- h2 stands for **Unique Operands** (η_2)
- N1 stands for **Total Operators** (N_1, the total count of all operators used)
- N2 stands for **Total Operands** (N_2, the total count of all operands used)
- Voc. stands for **Vocabulary** ($\eta = \eta_1 + \eta_2$, the total number of unique operators and operands)
- Len. stands for **Length** ($N = N_1 + N_2$, the total number of operator and operand occurrences)
- Calc. Len. stands for **Calculated Length** ($\hat{N} = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$, an estimated program length)
- Volume stands for **Volume** ($V = N \log_2 \eta$, the minimum number of bits required to encode the program)
- Diff. stands for **Difficulty** ($D = \frac{\eta_1}{2} \cdot \frac{N_2}{\eta_2}$, a measure of how hard it is to write or understand the program)

- Effort stands for **Effort** ($E = V \times D$, the required mental effort to develop the software).

Halstead metrics showed that computer.py required the highest mental effort to develop and understand, with an Effort score significantly higher than other modules (Volume 801.02, Effort 6511.51).

4.4 Cohesion

Table 2: Total Cohesion per Module/File

File Name	**Total Cohesion**
program/dice.py	0.0%
program/player.py	33.33%
program/menu.py	26.19%
program/dice_hand.py	66.67%
program/high_score.py	33.33%
program/game.py	55.0%
program/computer.py	22.92%

Table 2 shows us cohesion for each file. We can see here that the dice file has 0%. The file has only static methods.Most of the code is graded extremelly low which means that Single Responsibility Principle was violated.

Function	**Cohesion (%)**
`__init__`	75.00%
select_difficulty	25.00%
change_computer_name	12.50%
roll_dice	0.00%
hold_at_twenty	12.50%
scoring_turn	37.50%
keep_or_race	25.00%
take_turn	37.50%
check_valid_computer_name	12.50%
check_valid_difficulty	12.50%
get_total_score	12.50%
get_username	12.50%
Total Cohesion	**22.92%**

Table 3: Cohesion metrics for `program\computer.py` (Class: Computer)

This table provides us more detailed information about cohesion in class computer and how it could be broken apart for more cohesive ways.

5 Discussion

The results of the practical assignment, particularly the low code coverage and poor cohesion metrics (Section 3), provide a quantitative basis for reflecting on the efficacy of the chosen development methodologies and their impact on code sustainability.

5.1 Philosophies and Principles

Several philosophies and principles are relevant to the outcome, such as:

- DRY(Don't Repeat Yourself) - reduce repetition of information that is likely to change replacing it with abstractions
- KISS (Keep it simple stupid) - principle from U.S Navy that all choices should be extremelly simple.
- YAGNI (You Ain't Gonna Need It) - way of programming, where things are implemented only when they are required

Reflecting personal experience, my approach was aligned closely to KISS: it's preferable to spend extra time to make code shorter and simplify logic. However based on results from Section 3, project work abandoned such principles that led to unnessecary complexity.

5.2 Test Driven Development and cost of the proccess flaw

At its core Test Driven Development is about writing tests before the code. Such cycle leads to writing as little code as possible to pass the tests which corresponds to philosophies such as YAGNI and KISS.

5.2.1 Team's approach to the assignment and how TDD wasn't implemented

Our team's plan on doing assignment can be described in such steps:

1. Create UML and architecture
2. Assign each person a class.
3. Each person writes their code.
4. Person exchanges the code with other member and both of them unit test the other person's code.
5. After receiving tests and code back, member rewrites their code to pass written tests.
6. Integrate all classes altogether and then pass the assignment.

Seeing the steps, TDD is not implemented only in half: we started with writing the actual program and not the tests. Team misinterpreted the task and TDD overall, so we wouldn't merge any of the branches until all of them are rightfully tested. This failed applying one of the TDD's leverages: design guidance. Such development of the product leaded for a late introducing of problems. As shown in the **Table 6**, bugs that were discovered in post-release cost 10–25× more if they were taggled in early stages of development. The subsequent "sudden" realisation that "integration needs some time and changes in the code" was an alert of neglecting continuous integration.

Table 4: Cost to Fix a Defect Based on Time Introduced vs. Time Detected [3]

Cost to fix a defect		**Time detected**				
		Requirements	**Architecture**	**Construction**	**System test**	**Post-release**
Time Introduced	**Requirements**	1×	3×	5–10×	10×	10–100×
	Architecture	–	1×	10×	15×	25–100×
	Construction	–	–	1×	10×	10–25×

5.3 Functional Programming

During this project it was discovered that procedural paradigm can cause issues during testing phase, therefore using functional paradigm showed more benefits.

5.3.1 Pure Functions in Unit Testing

One of the classes, high_score.py had functional paradigm in it. Seeing the coverage in Section 3 we can see that only 1 line wasn't tested out of all program. It's one of the most tested files throughout the whole project. The coverage data (92%) and high Maintainability Index (81.58) reflect the benefits of this approach

As Functional Programming implements Pure functions, it has a great advantage in unit testing: no side effects in all functions which means same input has the same output all the time. It eliminates dependencies on mostly anything and additionally makes tests simple and very isolated which could be one of properties of clean coding.

5.3.2 Functional paradigm or conventional style

During writing the high_score.py file I encountered into conflict between my vision of functions for unit testing and flake8 view. Specifically, I preferred to pass input variables directly into functions rather than relying on stored class variables. While linters promote conventional OOP encapsulation, in the context of unit testing, the functional preference (requiring explicit input for every function) maximizes testability and readability, aligning with the programming philosophy KISS (Keep It Simple, Stupid) by making dependencies explicit.

5.4 Technical debt

Deadline and wrong approach to the project led the team to a technical debt – quick and simple resolving of the problem that worsened the quality of code and maintability of it. Due to such circumstances testing coverage got extremelly low as there wasn't that much time to retest everything.

6 Summary

This paper analyzed the sustainability of a software project through the lens of TDD and functional programming. The key findings are:

1. Methodology Matters: Partial adherence to TDD is insufficient. Writing unit tests before integration leads to drastical increases of problems in the code.

2. Quantitative Metrics reflect Quality: The low cohesion (22.92%) and coverage (9%) of the most complex module accurately predicted the difficulty the team faced in writing tests to it.

3. Functional Paradigm promotes Sustainability: The use of pure functions in `high_score.py` resulted in higher maintainability and testability compared to the state-heavy OOP modules.

Sustainable software development requires not just working code, but code that is modular, testable, and developed with a process that catches defects early.

7 Bibliography

[1] L. Vaillant, *What is good code?* Loup-vaillant.fr, 2015. Accessed: Dec. 6, 2025. [Online]. Available: https://loup-vaillant.fr/articles/good-code.

[2] *Coverage.py — coverage.py 7.12.0 documentation*, Readthedocs.io, 2025. Accessed: Dec. 5, 2025. [Online]. Available: https://coverage.readthedocs.io/en/7.12.0/.

[3] S. McConnell, *Code Complete*. Pearson Education, Jun. 2004, p. 24.

[4] *Teach yourself programming in ten years*, Norvig.com, 2025. [Online]. Available: https://norvig.com/21-days.html.

[5] J. Spolsky, *Things you should never do, part i*, Joel on Software, Apr. 2000. [Online]. Available: https://www.joelonsoftware.com/2000/04/06/things-you-should-never-do-part-i/.

A Reflections on Polyglot Programming

During the research phase that were on Canvas, I found an article [4] that was debating about books such as "Teach C++ in 24 hours" and how they mislead people into thinking that they will actually know C++ by the end of the book. The core argument is that coding quality and understanding is coming from years of practise, not days. This resonates with the quote: "A language that doesn't affect the way you think about programming is not worth knowing."

Reflecting on this, my mother said that in her university all task had requirement to write on different language. Additionaly, on current course when I had to chance to use Java, I decided to switch sides and write tasks in C/C++. That was a game changer and I incredibly enjoyed it, because I found lots of new information about manipulations with memory that i would have never knew if I used only high-level languages like python or java.

For functional paradigm part of this course, I really badly wanted to write on different language that is based on this paradigm. Unfortunately, my team didn't allow me to do that. Nevertheless, after spending few hours figuring out what is actually a functional paradigm in OOP Java, I got it. Maybe, it was easier to understand with Java. And functional paradigm changed my thoughts about programming overall. For second assignment I tried to code with same style, which was quite fun. And just because of that I discovered that functional paradigm is perfectly fitted with unit testing. Testing functions is way easier than testing code that has dependances on other functions.

B Sustainable development

Another article [5] that I found on Canvas really intrigued me while I was reading. I would say, this article exactly describes what I imagine sustainable development is mentioned. Article explains how throwing away the whole program just to rewrite it with more understandable way is a bad idea. It reminds me of my first task with functional paradigm: our team decided to refactor old assignment that we had from oop java course. We took my work, with which i was extremelly proud. We learned that sometimes, code is not just "messy," but fundamentally misaligned with new requirements. We encountered quite a few problems: first problem was that refacturing wasn't a gat idea for a tiny file with small classes that have 4 functions..

Nevertheless it was good experience of refacturing code without throwing everything away.